

Home SOC Lab:

Wazuh SIEM Deployment, Endpoint Detection & Attack Simulation

A self-built security operations lab covering SIEM deployment, endpoint detection engineering, and a live attack-detection exercise.

Angad Chahal

September 2026

1. Overview

I designed and built a home Security Operations Center (SOC) lab to gain hands-on experience with the tools and workflows used in real-world security monitoring. Using VirtualBox, I deployed Wazuh — an open-source SIEM (Security Information and Event Management) platform — and connected it to multiple endpoints across both Windows and Linux, then validated detection across several attack surfaces: file tampering, malware, suspicious process activity, and a live network-based attack.

The lab environment consisted of four virtual machines plus my physical Windows PC, all networked together to allow safe, isolated testing:

- Wazuh Server — the SIEM manager, indexer, and dashboard (deployed from the official OVA)
- Windows Endpoint (physical PC) — a monitored Windows agent
- Kali Linux — a second monitored Linux endpoint, later repurposed as the attacker machine
- Ubuntu Server ("Victim") — a dedicated, disposable target for the attack simulation

Every step below was performed hands-on: deploying infrastructure, editing configuration files directly, diagnosing and resolving real technical failures, and validating each detection with live evidence in the Wazuh dashboard.

Skills Demonstrated

- SIEM deployment and administration (Wazuh manager, indexer, dashboard)
- Virtual network design (host-only + NAT segmentation for lab isolation)
- Linux and Windows system administration (via CLI/SSH and PowerShell)
- Endpoint detection configuration: File Integrity Monitoring, Sysmon, Defender log forwarding
- Custom detection rule authoring (Wazuh XML ruleset)
- Offensive security basics: SSH brute-force simulation using Hydra
- Systematic troubleshooting: disk capacity failures, service crashes, and driver/networking issues

2. Lab Architecture & Network Design

Rather than following the guide's suggestion to use bridged networking, I designed a segmented lab network:

- Adapter 1 — Host-only (192.168.56.0/24): a private network connecting all lab VMs and my PC to the Wazuh server.
- Adapter 2 — NAT (10.0.2.0/24): gives each VM outbound internet access for updates and package installs, without exposing them to my home network.

Decision rationale: the guide I referenced suggested bridged networking, which is simpler to set up but places every lab VM directly on the home LAN. Because a later phase of this lab involved simulating an SSH brute-force attack with Hydra, I chose host-only + NAT instead, so that attack traffic never touches my real network, IP addresses stay stable across reboots, and the design leaves room to add a firewall/IDS layer later without re-architecting the network.

```
WAZUH Open Source Security Platform
https://wazuh.com
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:89:19:6e brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.56.101/24 metric 1024 brd 192.168.56.255 scope global dynamic eth0
        valid_lft 542sec preferred_lft 542sec
    inet6 fe80::a00:27ff:fe89:196e/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:48:72:af brd ff:ff:ff:ff:ff:ff
    altname enp0s8
    inet 10.0.2.15/24 metric 1024 brd 10.0.2.255 scope global dynamic eth1
        valid_lft 86342sec preferred_lft 86342sec
    inet6 fd00::a00:27ff:fe48:72af/64 scope global dynamic mngtnpaddr noprefixroute
        valid_lft 86345sec preferred_lft 14345sec
    inet6 fe80::a00:27ff:fe48:72af/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$ SSSSSS
```

Figure 1: Confirming the Wazuh server's IP addresses. eth0 (192.168.56.101) is the host-only lab network; eth1 (10.0.2.15) is the NAT interface used for internet access.

```
C:\Users\andyc>ping 192.168.56.101

Pinging 192.168.56.101 with 32 bytes of data:
Reply from 192.168.56.101: bytes=32 time=1ms TTL=127
Reply from 192.168.56.101: bytes=32 time<1ms TTL=127
Reply from 192.168.56.101: bytes=32 time<1ms TTL=127
Reply from 192.168.56.101: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.56.101:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\andyc>
```

Figure 2: Verifying network connectivity between the Windows host and the Wazuh server before installing any agents.

3. Wazuh Server Deployment & Agent Enrollment

I deployed Wazuh 4.14.7 from the official OVA into VirtualBox, applying the network configuration above, then enrolled two initial agents: my Windows PC and a Kali Linux VM.

"I downloaded the Wazuh OVA and imported it into VirtualBox, using a Host-only + NAT network so the lab stays isolated. After booting the VM and confirming its IP (192.168.56.101), I logged into the dashboard from my Windows PC and deployed a Windows agent using the generated PowerShell command. I started the agent service, and the dashboard now shows it as Active."

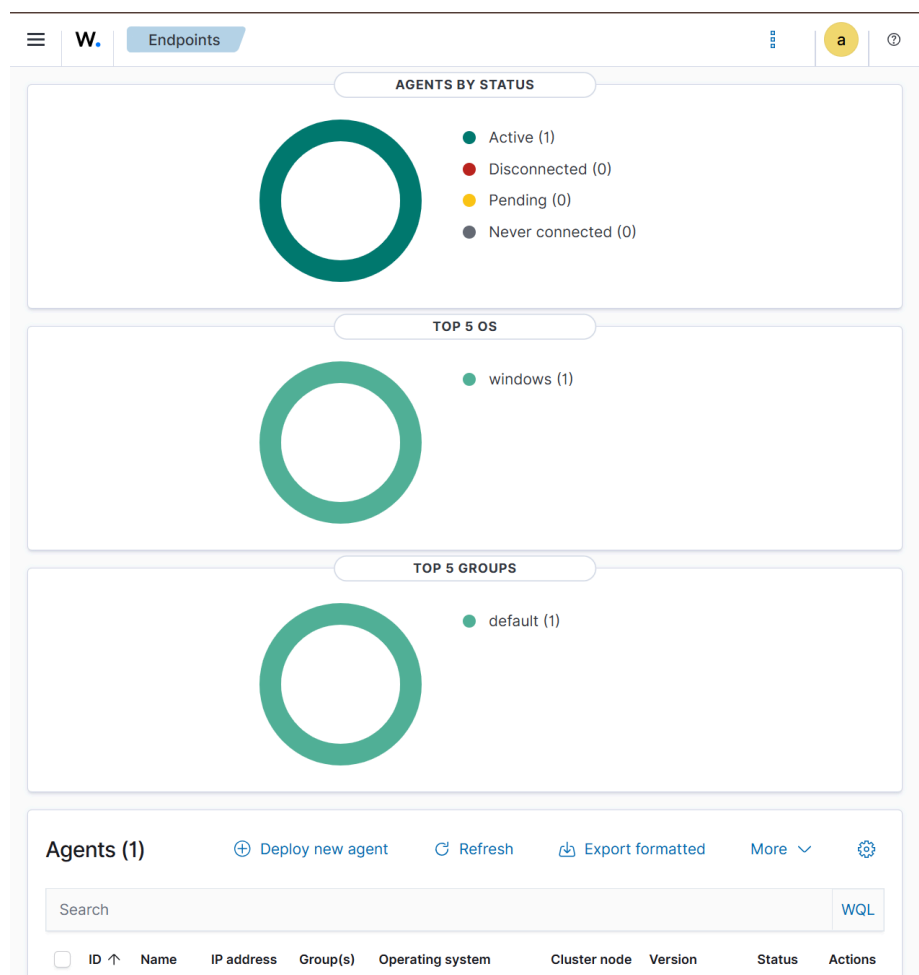


Figure 3: Wazuh Endpoints dashboard showing the Windows agent successfully enrolled and reporting as Active.

Troubleshooting note — server disk capacity

Partway through testing, alerts stopped appearing in the dashboard entirely. Investigation traced this to the Wazuh server's virtual disk filling to 100% capacity (the default OVA allocation of 25 GB was undersized for a running indexer). I diagnosed this by SSH'ing into the server and checking disk usage directly, then resolved it by:

- Enlarging the VirtualBox virtual disk from 25 GB to 60 GB
- Expanding the Linux partition and filesystem to use the new space (growpart + xfs_growfs)
- Clearing an index read-only lock the indexer had set when the disk filled, and restarting the Wazuh services in the correct dependency order (indexer → filebeat → dashboard)

This was the most significant infrastructure issue in the project and a good example of real SOC-analyst-adjacent troubleshooting: recognizing that a monitoring pipeline had silently stopped, tracing it to root cause, and fixing it without losing existing data.

4. File Integrity Monitoring (FIM)

I configured Wazuh's File Integrity Monitoring module on both endpoints to detect unauthorized file changes in real time — a control that maps directly to compliance requirements such as PCI-DSS and detects tampering, malware drops, and unauthorized modifications.

4.1 Windows Endpoint

I added a <directories> entry to the agent's syscheck configuration with real-time monitoring and change reporting enabled, targeting a dedicated test folder.

"The agent's ossec.conf contains a <syscheck> block that defines which files Wazuh monitors. I added a <directories> entry for my test folder with real-time monitoring enabled, so any file change there triggers an alert."

```
<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories recursion_level="0" restrict="regedit.exe|system.ini|win.ini">%WINDIR%</directories>

  <directories recursion_level="0" restrict="at.exe|attrib.exe|cacls.exe|cmd.exe|eventcreate.exe|ftp.exe|lsass.exe|net.exe|net1.exe|netsh.exe
$|reg.exe|regedt32.exe|regsvr32.exe|runas.exe|sc.exe|schtasks.exe|sethc.exe|subst.exe">%WINDIR%\SysNative</directories>
  <directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe">%WINDIR%\SysNative\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe">%WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs">%WINDIR%\SysNative</directories>

  <!-- 32-bit programs. -->
  <directories recursion_level="0" restrict="at.exe|attrib.exe|cacls.exe|cmd.exe|eventcreate.exe|ftp.exe|lsass.exe|net.exe|net1.exe|netsh.exe
$|reg.exe|regedt32.exe|regsvr32.exe|runas.exe|sc.exe|schtasks.exe|sethc.exe|subst.exe">%WINDIR%\System32</directories>
  <directories recursion_level="0">%WINDIR%\System32\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe">%WINDIR%\System32\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe">%WINDIR%\System32\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs">%WINDIR%\System32</directories>

  <directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup</directories>

  <directories check_all="yes" report_changes="yes" realtime="yes">C:\Users\andyc\Documents\FIM_Test</directories>
```

Figure 4: FIM configuration added to the Windows agent's ossec.conf, enabling real-time monitoring with checksum and change reporting.

Validation: creating, editing, and deleting a test file produced the expected alert sequence:

"I configured File Integrity Monitoring on the Windows endpoint to watch a test folder in real time. Creating, modifying, and deleting a file generated Wazuh alerts: level 5 for the file being added, and level 7 for modifications and deletions. This confirmed that FIM works and the alerts reach the dashboard."

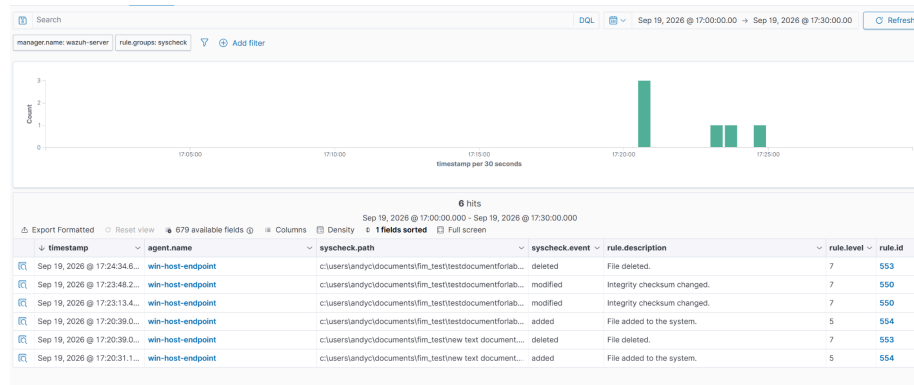


Figure 5: Wazuh dashboard confirming FIM alerts for file added, modified, and deleted events on the Windows endpoint.

4.2 Kali Linux Endpoint

The same configuration was replicated on the Kali endpoint to confirm cross-platform consistency.

```
<sca>
<enabled>yes</enabled>
<scan_on_start>yes</scan_on_start>
<interval>12h</interval>
<skip_nfs>yes</skip_nfs>
</sca>

<!-- File integrity monitoring -->
<syscheck>
<disabled>no</disabled>

<!-- Frequency that syscheck is executed default every 12 hours -->
<frequency>43200</frequency>

<scan_on_start>yes</scan_on_start>

<!-- Directories to check (perform all possible verifications) -->
<directories>/etc,/usr/bin,/usr/sbin</directories>
<directories>/bin,/sbin,/boot</directories>

<directories check_all="yes" report_changes="yes" realtime="yes">/home/kali/FIM_Test</directories>

<!-- Files/directories to ignore -->
<ignore>/etc/mtab</ignore>
<ignore>/etc/hosts.deny</ignore>
<ignore>/etc/mail/statistics</ignore>
<ignore>/etc/random-seed</ignore>
<ignore>/etc/random.seed</ignore>
<ignore>/etc/adjtime</ignore>
<ignore>/etc/httpd/logs</ignore>
<ignore>/etc/utmpx</ignore>
<ignore>/etc/wtmpx</ignore>
<ignore>/etc/cups/certs</ignore>
<ignore>/etc/dumpdates</ignore>
<ignore>/etc/svc/volatile</ignore>

<!-- File types to ignore -->
<ignore type="sregex">.log$|.swp$</ignore>

<!-- Check the file, but never compute the diff -->
<nodiff>/etc/ssl/private.key</nodiff>
```

Figure 6: FIM configuration on the Kali Linux agent, monitoring /home/kali/FIM_Test in real time.

"I deployed a Wazuh agent on a Kali Linux VM and configured File Integrity Monitoring on a test folder. Creating, modifying, and deleting a file produced level 5 and level 7 alerts in the dashboard, matching the Windows endpoint."

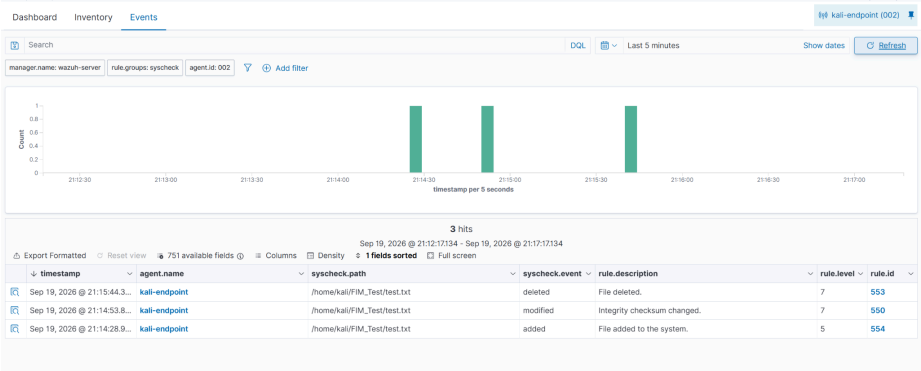


Figure 7: FIM alerts on the Kali endpoint, matching the detection behavior seen on Windows.

5. Windows Defender Log Integration

To centralize antivirus telemetry in the SIEM, I configured the Windows agent to forward Microsoft Defender's event log to Wazuh, then validated it using the industry-standard EICAR test file (a harmless string recognized by all major antivirus engines as a test signature).

```

<!-- Agent buffer options -->
<client_buffer>
  <disabled>no</disabled>
  <queue_size>5000</queue_size>
  <events_per_second>500</events_per_second>
</client_buffer>

<!-- Log analysis -->
<localfile>
  <location>Application</location>
  <log_format>eventchannel</log_format>
</localfile>

<localfile><location>Microsoft-Windows-Windows Defender/Operational</location><log_format>eventchannel</log_format></localfile>

<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
  <query>Event/System[EventID != 5145 and EventID != 5156 and EventID != 5447 and
    EventID != 4656 and EventID != 4658 and EventID != 4663 and EventID != 4660 and
    EventID != 4670 and EventID != 4690 and EventID != 4703 and EventID != 4907 and
    EventID != 5152 and EventID != 5157]</query>
</localfile>

<localfile>
  <location>System</location>
  <log_format>eventchannel</log_format>
</localfile>

</localfile>

```

Figure 8: Defender log source added to the Wazuh agent configuration — a `<localfile>` block reading the Microsoft Defender Operational event channel.

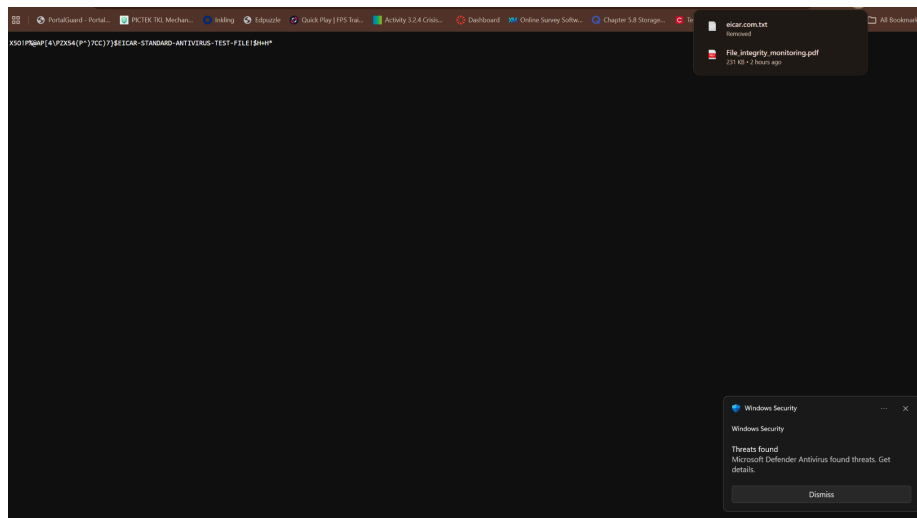


Figure 9: Windows Defender detecting and removing the EICAR test file, confirming the antivirus layer is active.

6. Sysmon Deployment & Custom Detection Rule

Default Windows event logs provide limited visibility into process activity. To close this gap, I installed Sysmon (Microsoft Sysinternals) with the community-maintained SwiftOnSecurity configuration and forwarded its Operational log to Wazuh.

A key finding during testing was that Wazuh's dashboard only surfaces alerts at level 3 or higher, but most default Sysmon events (e.g., normal process launches) are scored below that threshold and were invisible in the dashboard despite being correctly ingested. I resolved this by authoring a custom detection rule in `local_rules.xml` that elevates Sysmon Event ID 1 (process creation) to level 8.

```

    <port>1514</port>
    <protocol>tcp</protocol>
  </server>
  <config-profile>windows, windows10</config-profile>
  <crypto_method>aes</crypto_method>
  <notify_time>20</notify_time>
  <time-reconnect>60</time-reconnect>
  <auto_restart>yes</auto_restart>
  <enrollment>
    <enabled>yes</enabled>
    <agent_name>win-host-endpoint</agent_name>
  </enrollment>
</client>

<!-- Agent buffer options -->
<client_buffer>
  <disabled>no</disabled>
  <queue_size>5000</queue_size>
  <events_per_second>500</events_per_second>
</client_buffer>

<!-- Log analysis -->
<localfile>
  <location>Application</location>
  <log_format>eventchannel</log_format>
</localfile>

<localfile><location>Microsoft-Windows-Windows Defender/Operational</location><log_format>eventchannel</log_format></localfile>
<localfile><location>Microsoft-Windows-Sysmon/Operational</location><log_format>eventchannel</log_format></localfile>

<localfile>
  <location>Security</location>
  <log_format>eventchannel</log_format>
</localfile>

```

Figure 10: Sysmon's Operational event channel added to the Windows agent configuration (highlighted line).

"I installed Sysmon with the SwiftOnSecurity configuration and added its Operational event channel to the agent. Because routine Sysmon events score too low for the dashboard, I wrote a custom rule (ID 100100) that raises Sysmon Event ID 1 (process creation) to level 8. Launching programs on the endpoint generated these alerts, confirming that Sysmon telemetry reaches Wazuh."

```

GNU nano 8.3 /var/ossec/etc/rules/local_rules.xml
<!-- Local rules -->
<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>97160</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

</group>

<group name="sysmon,">
  <rule id="100100" level="8"><if_sid>61603</if_sid><description>Sysmon - Process created (custom rule)</description></rule></group>

```

Figure 11: Custom Wazuh detection rule (ID 100100) authored in `local_rules.xml` to elevate Sysmon process-creation events to a visible alert level.

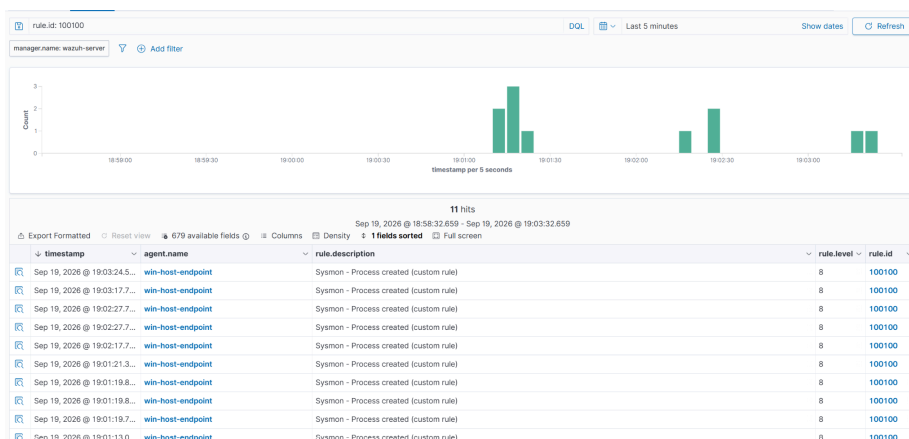


Figure 12: Validation — the custom rule firing at level 8 each time a process was launched on the Windows endpoint.

7. SSH Brute-Force Attack Simulation & Detection

To validate end-to-end detection of an active attack — not just passive log collection — I built a dedicated, disposable Ubuntu Server VM as an attack target, then used Hydra (from the Kali VM) to run a dictionary-based SSH brute-force attack against it, and confirmed the full kill chain was visible in Wazuh.

I deliberately built a separate victim machine rather than attacking the Wazuh server or Windows endpoint directly, to keep the SIEM's own infrastructure safe from disruption and to mirror how a real SOC segregates monitoring infrastructure from attack surfaces.

"Installed hydra and made a small password list, also installed a second Ubuntu VM as test subject."

```
(kali@kali)~$ sudo apt update && sudo apt install -y hydra
[sudo] password for kali:
Ign:1 http://mirror.us.cdn-perfprod.com/kali kali-rolling InRelease
Ign:1 http://mirror.us.cdn-perfprod.com/kali kali-rolling InRelease
Ign:1 http://mirror.us.cdn-perfprod.com/kali kali-rolling InRelease
Err:1 http://mirror.us.cdn-perfprod.com/kali kali-rolling InRelease
       Could not connect to mirror.us.cdn-perfprod.com:80 (192.210.198.18). - connect (113: No route to host)
       Unable to connect to mirror.us.cdn-perfprod.com:http:
All packages are up to date.
Warning: Failed to fetch http://http.kali.org/kali/dists/kali-rolling/InRelease Unable to connect to mirror.us.cdn-perfprod.com:http:
Warning: Some index files failed to download. They have been ignored, or old ones used instead.
hydra is already the newest version (9.7-1).
hydra set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(kali@kali)~$ printf 'wrongpass1\nadmin123\nletmein\nPassword123\nqwerty\n123456\n' > /home/kali/passwords.txtS
```

Figure 13: Installing Hydra and preparing a candidate password list on the Kali attacker VM.

```
$ cat /home/kali/passwords.txt
wrongpass1
admin123
letmein
changeme
Password123
qwerty
123456

(kali@kali)~$ hydra -l vboxuser -P /home/kali/passwords.txt ssh://192.168.56.105 -t 4 -V
hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organization
or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-20 00:10:56
[DATA] max 4 tasks per 1 server, overall 4 tasks, 7 login tries (1:1/p:7), ~2 tries per task
[DATA] attacking ssh://192.168.56.105:22/
[ATTEMPT] target 192.168.56.105 - login "vboxuser" - pass "wrongpass1" - 1 of 7 [child 0] (0/0)
[ATTEMPT] target 192.168.56.105 - login "vboxuser" - pass "admin123" - 2 of 7 [child 1] (0/0)
[ATTEMPT] target 192.168.56.105 - login "vboxuser" - pass "letmein" - 3 of 7 [child 2] (0/0)
[ATTEMPT] target 192.168.56.105 - login "vboxuser" - pass "changeme" - 4 of 7 [child 3] (0/0)
[22][ssh] host: 192.168.56.105 login: vboxuser password: changeme
1 of 1 target successfully completed, 1 valid password found
hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-20 00:11:03

(kali@kali)~$
```

Figure 14: Hydra successfully identifying the correct SSH credential after cycling through the password list against the victim VM.

Wazuh's detection engine logged the individual failed authentication attempts and then automatically correlated them into a single high-severity brute-force alert once the failure threshold was crossed within its detection window — this correlation, rather than the individual failures, is the signal a SOC analyst would actually act on.

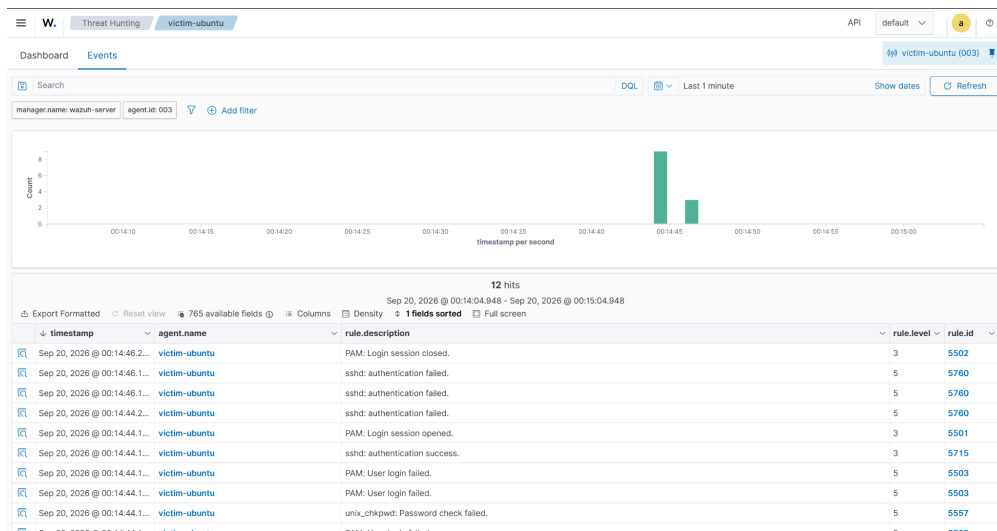


Figure 15: The full attack sequence in Wazuh — repeated sshd authentication failures, followed by a successful authentication and session opened.

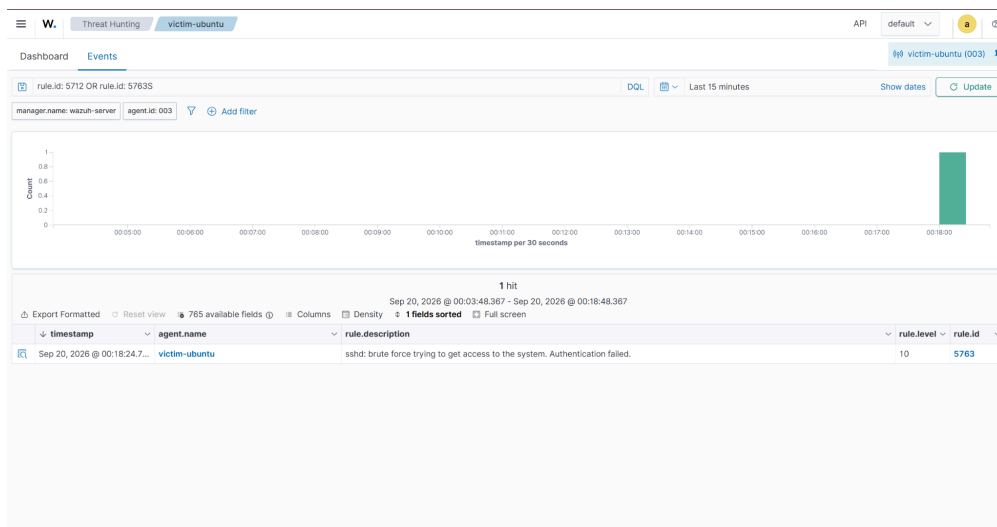


Figure 16: Wazuh's correlation engine firing rule 5763 (level 10) — "sshd: brute force trying to get access to the system" — after repeated failures from the same source.

"I used Hydra from the Kali attacker VM to run a dictionary-based SSH brute-force attack against a dedicated Ubuntu victim VM. Wazuh logged each failed authentication attempt (rule 5760, level 5) and, after enough failures within its detection window, triggered its built-in correlation rule (rule 5763, level 10): 'sshd: brute force trying to get access to the system.' The attack ultimately succeeded, and Wazuh also logged the successful authentication (rule 5715) and session opening (rule 5501), demonstrating the full attack lifecycle from initial reconnaissance through compromise."

8. Results Summary

Component	Status	Evidence
Wazuh server deployment	Complete	Dashboard access, agent enrollment

Component	Status	Evidence
Network segmentation (host-only + NAT)	Complete	ip a, connectivity tests
Windows agent	Complete	Active status in dashboard
Kali Linux agent	Complete	Active status in dashboard
File Integrity Monitoring (Windows)	Complete	Level 5/7 alerts on file change
File Integrity Monitoring (Kali)	Complete	Level 5/7 alerts on file change
Windows Defender log integration	Complete	EICAR detection, rules 62123/62124
Sysmon + custom detection rule	Complete	Custom rule 100100 firing on process creation
SSH brute-force simulation & detection	Complete	Rules 5760, 5763, 5715, 5501
Disk-capacity incident response	Complete	Diagnosed & resolved without data loss

9. Known Limitations

Two additional integrations were attempted but not completed within the project timeline, due to environment-level issues rather than configuration errors on my part. I've documented both, since diagnosing and clearly bounding an unresolved issue is itself a useful SOC skill.

Suricata IDS

Installed Suricata and Npcap on the Windows endpoint to add network intrusion detection. The engine crashed on startup (access violation) when initializing packet capture. Diagnosis traced this to the Npcap driver being installed but not registered as a running Windows service — a known Suricata/Windows compatibility issue rather than a configuration mistake.

VirusTotal Integration

Configured Wazuh's VirusTotal integration with a valid API key to automatically check files flagged by FIM against VirusTotal's reputation database. The manager confirmed the integration loaded correctly, but the outbound call failed. Root-cause diagnosis (via curl and ping directly from the server) revealed that the Wazuh VM's NAT network adapter had stopped routing to its own gateway — a VirtualBox networking fault, confirmed independently of Wazuh and unresolved by an adapter reset.

10. Conclusion

This project took me through the full lifecycle of standing up a SIEM: infrastructure deployment, network design, multi-platform agent management, detection engineering (both using built-in rules and authoring my own), and validating detections against real attacker tooling. Beyond the successful integrations, the two issues I hit and diagnosed but didn't fully resolve — a full disk silently breaking the alert pipeline, and a driver/networking fault on Windows and in VirtualBox — reflect the kind of real-world troubleshooting a SOC analyst or security engineer encounters regularly, and I documented each with the evidence and reasoning behind my diagnosis rather than treating them as failures.